

OOPJ Day 15

Shivam Zanjurne · Wed, 23 Sept 26

Table of Contents

1. Stream API Recap
2. Collectors and Wildcards
3. Synchronous vs. Multithreaded Execution
4. Creating Threads in Java
5. Worker Threads vs. Daemon Threads
6. Thread Priority
7. Context Switching and Performance
8. Synchronization and Inter-thread Communication
9. Java Date API and Closing Guidance

Stream API Recap

- Stream API is used when you want to process a collection of data without manually iterating over it
- Two categories of operations on a stream:

Intermediate Operations

- Return another stream object, enabling method chaining (the "pipeline")
- Examples: `filter()`, `map()`, `sorted()`
- Processing does **NOT** occur when intermediate operations are called; they are **lazy**
- Each intermediate operation returns a new stream reference:

```
stream.filter( ... ).map( ... ).sorted( ... ) // still no processing yet
```

Terminal Operations

- Trigger actual processing of the entire pipeline
- Examples: `collect()`, `forEach()`, `count()`, `reduce()`
- Once a terminal operation is called, the stream is **consumed** and cannot be reused

```
List<String> result = myList.stream()
    .filter(s → s.startsWith("A")) // intermediate
    .map(String::toUpperCase)      // intermediate
    .collect(Collectors.toList()); // terminal - triggers processing
```

- Stream objects are **single-use**: once consumed by a terminal operation, you must open a new stream to process again

```
Stream<String> s = myList.stream();
s.collect(Collectors.toList()); // OK
s.collect(Collectors.toList()); // IllegalStateException - stream already consumed
```

Collectors and Wildcards

Collectors.toList() and Collectors.toSet()

- Convert the result of a stream pipeline into a collection object
- `toList()` returns an `ArrayList`; `toSet()` returns a `HashSet`

```
List<Book> fiction = bookList.stream()
    .filter(b → b.getGenre() == Genre.FICTION)
    .collect(Collectors.toList());
```

Collectors.groupingBy()

- Groups stream elements into a `Map` based on a classifier `Function`
- **Key** = the grouping criterion; **Value** = `List` of matching elements

```
Map<Genre, List<Book>> byGenre = bookList.stream()
    .collect(Collectors.groupingBy(Book::getGenre));
// Key: FICTION, THRILLER, etc. → Value: List of books in that genre
```

Collectors.partitioningBy()

- Splits stream elements into exactly **two** groups using a `Predicate`
- Returns a `Map<Boolean, List>`: `true` key = matches predicate, `false` key = does not match

```
Map<Boolean, List<Book>> partitioned = bookList.stream()
    .collect(Collectors.partitioningBy(b → b.getPrice() > 500));
// true → books costing more than ₹500
// false → books costing ₹500 or less
```

Other Terminal Operations

- `allMatch()`, `anyMatch()`, `noneMatch()`: test conditions across the stream
- `max()`, `min()`: find extremes using a `Comparator`
- `count()`: returns the number of elements

Wildcards in Generics

- When defining a generic class or method, you can constrain the type parameter using wildcards

- Basic generic class:

```
class Data<T> {
    T value;
}
```

Upper Bounded Wildcard (`extends`)

- Syntax: `? extends Base` or `T extends Base`
- Meaning: `T` can be `Base` itself, or any class that inherits from `Base`

```
class Data<T extends Base> {
    T value;
}
// Accepts: Base, Derived (if Derived extends Base)
// Rejects: Object, or any unrelated class
```

Lower Bounded Wildcard (`super`)

- Syntax: `? super Base` or `T super Base`
- Meaning: `T` can be `Base` itself, or any parent/ancestor of `Base` (up to `Object`)

```
// If hierarchy is: Object → Base → Derived
// ? super Derived accepts: Derived, Base, Object
// ? super Base    accepts: Base, Object
```

- `partitioningBy` uses a **lower bounded** wildcard predicate:

```
Collectors.partitioningBy(Predicate<? super T> predicate)
// For a stream of Book, predicate can be Predicate<Book> or Predicate<Object>
```

- `groupingBy` uses an **upper bounded** wildcard on the return type:

```
Collectors.groupingBy(Function<? super T, ? extends K> classifier)
// The return type K (e.g., Genre) can be Genre or any subtype of Genre
```

Key rule: `extends` = current class and its **derived** classes; `super` = current class and its **parent** classes

Synchronous vs. Multithreaded Execution

The Blocking Problem

- By default, Java (and all programs) execute statements **synchronously**: top to bottom, one at a time
- A method with an infinite loop will block all subsequent code indefinitely:

```
static void increment() {
    while (true) {
        System.out.println("Increment");
    }
}

static void decrement() {
    while (true) {
        System.out.println("Decrement");
    }
}

public static void main(String[] args) {
    increment(); // blocks forever - decrement() is never reached
    decrement();
    while (true) {
        System.out.println("Main");
    }
}
```

- **Output:** only "Increment" printed forever; "Decrement" and "Main" never execute
- This is called a **blocking call**: execution is stuck until the current method returns

Why Multitasking is Necessary

- Modern operating systems (Windows, Linux) must run multiple tasks simultaneously
- A **task (process)** = a single program execution loaded into memory with its own allocated resources
- Example: in Microsoft Word, typing and background spell-checking happen at the same time
 - These are not two separate programs; they are two **threads** within the same process

What is a Thread?

- A thread is an independent path of execution that is provided a separate time slice by the CPU
- **Formal definition:** a thread is an independent path of execution provided a separate time slice by the CPU
- A single processor can only execute one instruction at a time, yet appears to run many things simultaneously
- This illusion is created by **time slicing**: the OS rapidly switches between threads, giving each a small window of CPU time

Time Slicing Analogy

- Imagine 200 students in a classroom; the instructor (CPU) can only talk to one student at a time
- Instead of dedicating all attention to one student until they are done, the instructor rotates rapidly
- Each student gets a short turn (time slice), creating the illusion that everyone is being attended to simultaneously
- This is exactly how the OS manages threads via **round-robin scheduling**

Process vs. Thread

	Process	Thread
Memory	Independently allocated	Shares parent process memory
Resources	Own CPU cycle and resources	Borrows from parent process
Termination	Independent	Parent dies → all threads die
Creation overhead	High	Low

- Thread always belongs to a process; if the parent process terminates, all its threads terminate
- A thread terminating does **not** affect the parent process

Creating Threads in Java

Technique 1: Extending the Thread Class

```
class IncrementThread extends Thread {
    @Override
    public void run() {
        while (true) {
            System.out.println("Increment");
            try {
                Thread.sleep(100); // pause for 100ms, release CPU
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

- The `Thread` class is in `java.lang` and provides methods: `start()`, `stop()` (deprecated), `setPriority()`, `getName()`, `isAlive()`, `isDaemon()`, `join()`, `interrupt()`, etc.
- Override `run()` to define what the thread should do during its time slice

- The life of the thread is governed by the life of `run()` : when `run()` returns, the thread is destroyed

Starting a Thread

```
IncrementThread incThread = new IncrementThread();  
// Creating the object only allocates it in memory - no thread exists yet  
  
incThread.start();  
// start() does TWO things:  
// 1. Allocates resources (memory, CPU cycle) under the current process  
// 2. Requests a separate time slice from the JVM/OS  
// Internally, start() calls run() in that new time slice
```

Critical: calling `incThread.run()` directly is a **synchronous** call — no new thread is created, no time slice is allocated. It behaves exactly like the original blocking `increment()` method.

Technique 2: Implementing the `Runnable` Interface

```
class DecrementThread implements Runnable {  
    @Override  
    public void run() {  
        while (true) {  
            System.out.println("Decrement");  
            try {  
                Thread.sleep(100);  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

- `DecrementThread` is **NOT** a thread; it is a `Runnable` — it defines *what task* should be executed
- It has no thread-control methods (no `start()`, no `setPriority()`, no `isAlive()`, etc.)
- To actually run it as a thread, wrap it in a `Thread` object:

```
DecrementThread target = new DecrementThread();  
Thread decThread = new Thread(target); // pass the Runnable as the task  
decThread.start();                     // now a separate time slice is allocated
```

Key Difference Between the Two Techniques

- `IncrementThread` is a `Thread` : it can be started, stopped, prioritized directly
- `DecrementThread` has a `run` method but is not a `Thread` : it must be assigned to a `Thread` object
- Internally, `Thread` itself implements `Runnable` — so extending `Thread` just overrides the same `run()` method

Why Prefer Runnable Over Extending Thread ?

1. **Inheritance is blocked:** Java does not support multiple inheritance; extending `Thread` prevents extending any other class
 - Example: a `SpellChecker` thread that also needs to extend `GrammarChecker` cannot do so if it extends `Thread`
2. **No behavior change:** extending a class is appropriate when you want to change its default behavior; since we are not changing `Thread`'s behavior, inheritance is semantically wrong
3. **Same result:** implementing `Runnable` achieves the same outcome with none of the drawbacks

Technique 3: Lambda Expression (Preferred for Simple Tasks)

- `Runnable` is a **functional interface** (one abstract method: `run()`), so it supports lambda expressions:

```
Runnable showRunnable = () -> {
    while (true) {
        System.out.println("Show");
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
};

Thread showThread = new Thread(showRunnable);
showThread.start();
```

- No separate class file is created (unlike anonymous classes)
- Inline and concise; preferred for short, one-off thread tasks

Anonymous Class Runnable (Pre-Lambda Approach)

```
Runnable testRun = new Runnable() {
    @Override
    public void run() {
        System.out.println("Anonymous class thread");
    }
};

Thread tm = new Thread(testRun);
tm.start();
// This creates a separate .class file (Program$1.class) - less clean than lambda
```

Thread.sleep() Explained

- Tells the JVM/OS: "I have nothing to do right now; give my CPU time to someone else"
- The thread does **not** hold the CPU during sleep; it voluntarily yields its time slice

- **Queue analogy:** you are standing in a queue and receive a phone call; you step out of the queue temporarily, letting the next person be served; when your call ends, you re-enter the queue

```
try {
    Thread.sleep(100);    // sleep for 100 milliseconds
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

- Always requires a `try-catch` because `sleep()` throws `InterruptedException`

Running Three Threads Simultaneously

```
public static void main(String[] args) {
    IncrementThread incThread = new IncrementThread();
    incThread.start();                                // Thread 1: Increment

    DecrementThread target = new DecrementThread();
    Thread decThread = new Thread(target);
    decThread.start();                                // Thread 2: Decrement

    Runnable showRunnable = () -> {
        while (true) {
            System.out.println("Show");
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {}
        }
    };
    Thread showThread = new Thread(showRunnable);
    showThread.start();                                // Thread 3: Show

    // Main itself is Thread 4 (invoked by JVM as a thread)
    while (true) {
        System.out.println("Main");
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {}
    }
}
// Output: Increment, Decrement, Show, Main interleaved simultaneously
```

Worker Threads vs. Daemon Threads

The Two Types

- **Worker thread:** a regular thread; JVM will **not** shut down while any worker thread is alive

- **Daemon thread:** a background/service thread; JVM shuts down when all worker threads finish, regardless of how many daemon threads are still running

JVM Shutdown Rule

As long as at least **one worker thread** is running, the JVM will not terminate — irrespective of the number of daemon threads still executing.

Real-World Analogy

- Word document has two background activities:
 - **Spell check = daemon thread:** if you close Word, it does not wait for spell check to finish all 150 pages
 - **File save (30 seconds) = worker thread:** if you close Word mid-save, it waits for the save to complete before shutting down

Default Behavior

- All threads in Java are created as **worker threads** by default
- The `main()` function itself is invoked as a worker thread by the JVM

Marking a Thread as Daemon

```
Thread decThread = new Thread(target);
decThread.setDaemon(true);    // must be called BEFORE start()
decThread.start();
```

Demonstration: Life of the Program

```
// IncrementThread: runs for 100 iterations (worker thread)
// DecrementThread: runs for 200 iterations (worker thread by default)

// Sequence:
// 1. main() starts           (worker thread 1)
// 2. incThread starts        (worker thread 2)
// 3. decThread starts        (worker thread 3)
// 4. main() finishes         → worker thread 1 dies; JVM still alive (2 workers remain)
// 5. incThread finishes       → worker thread 2 dies; JVM still alive (1 worker remains)
// 6. decThread finishes       → last worker thread dies; JVM shuts down
```

```
// Now make decThread a daemon:
decThread.setDaemon(true);

// Sequence:
// 1. main() starts      (only worker thread)
// 2. incThread starts   (worker thread)
// 3. decThread starts   (DAEMON thread)
// 4. main() finishes    → 1 worker remains (incThread)
// 5. incThread finishes → last worker dies; JVM shuts down immediately
//    decThread is killed mid-execution – no guarantee it completes
```

Why `stop()` is Deprecated

- `Thread.stop()` abruptly kills a thread mid-execution
- **Risk:** if a thread is writing to a file and `stop()` is called, the file may be half-written and corrupted
- Equivalent to pulling the power cable from a desktop PC

Graceful Shutdown with a Boolean Flag

- **Correct approach:** use a shared boolean flag to signal the thread to exit its `run()` loop naturally

```
class MainFrame extends JFrame {
    boolean runningStatus = true;    // shared flag

    // In window closing handler:
    runningStatus = false;           // signal threads to stop

    // In the rectangle thread's run():
    Runnable showRectangle = () → {
        while (frm.runningStatus) { // controlled loop, not infinite
            // draw rectangle logic
            try {
                Thread.sleep(25);
            } catch (InterruptedException e) {}
        }
        // run() returns naturally → thread terminates cleanly
    };
}
```

- When `runningStatus` becomes `false`, the `while` loop exits, `run()` returns, and the thread terminates gracefully
- No abrupt kill, no resource leak, no file corruption

`System.exit()` Warning

- `System.exit()` abruptly terminates the JVM — equivalent to pulling the power cable
- Garbage collection, resource cleanup, and background threads are all skipped
- Should **never** be used in production code; always let threads terminate naturally

Thread Priority

```
Thread decThread = new Thread(target);
decThread.setPriority(Thread.MAX_PRIORITY); // value 10
// Priority range: 1 (MIN) to 10 (MAX), default is 5 (NORM_PRIORITY)
```

- Higher priority = more frequent CPU turns (e.g., every 25 threads instead of every 200)
- Priority is a **request** to the JVM; the JVM may or may not honor it
- All modern OSes use **preemptive scheduling**: no thread can monopolize the CPU
 - **Non-preemptive** (old) systems allowed starvation: one thread could block the CPU indefinitely
 - **Preemptive** systems have a "bouncer": if a thread refuses to yield, the OS forcibly reclaims the CPU

Re-starting a Thread

- You **cannot** call `start()` on the same thread object twice; it throws `IllegalStateException`
- To run the same logic again, create a **new** `Thread` object:

```
// WRONG:
decThread.start(); // first time OK
decThread.start(); // throws IllegalStateException

// CORRECT:
Thread decThread2 = new Thread(target); // new Thread object, same or new Runnable
decThread2.start();
```

Context Switching and Performance

What is Context Switching?

- The OS gives each thread only **~20ms** of CPU time per turn (1/50th of a second)
- When a thread's time slice expires, the OS saves its state and switches to the next thread
- When the thread gets its turn again, it resumes exactly where it left off
- This save-and-restore cycle is called a **context switch**

What is a "Context"?

- A **context** = the complete state of a thread at a given moment:
 - The last instruction executed (memory address / program counter)
 - The state of all CPU registers and local variables
- Each thread is responsible for saving and loading its **own** context (not the OS)

- **Classroom analogy:** 200 students reading a book aloud, 20 seconds each
 - Each student must remember their own page and line number
 - The instructor (OS) cannot track 200 bookmarks simultaneously
 - Each student saves their bookmark (context) when their turn ends and loads it when their turn resumes

Context Switch Steps

1. Thread T1's time slice expires
2. T1 saves its context (last instruction, register values)
3. OS transfers control to T2
4. T2 loads its previously saved context
5. T2 resumes execution from where it left off

Performance vs. Throughput Tradeoff

- **Performance** (individual task speed): **decreases** with multithreading
 - T1 needs 2,000ms total; with 100 context switches at ~2ms each = 200ms overhead
 - Total time for T1 becomes 2,200ms instead of 2,000ms
- **Throughput** (total work done): **increases** with multithreading
 - Without multithreading: downloading a file takes 10 minutes; you can do nothing else
 - With multithreading: download takes 12 minutes, but you also wrote a letter in that time
 - More total work accomplished in the same wall-clock time

Summary

Metric	Single-threaded	Multi-threaded
Individual task time	Faster	Slower (context switch overhead)
Total throughput	Lower	Higher
Responsiveness	Blocked	Responsive

Synchronization and Inter-thread Communication

The Race Condition Problem

- When two threads share and modify the same data, context switching mid-operation produces **inconsistent results**
- **Bank account example:**
 - Account 7774 has ₹10,000 balance

- Thread 1 (deposit ₹5,000) and Thread 2 (withdrawal ₹5,000) run simultaneously

```
// Thread 1 - Deposit
int balance = getBalance(7774); // reads 10,000
balance = balance + 5000; // computes 15,000
// ← CONTEXT SWITCH HAPPENS HERE
setBalance(7774, balance); // sets 15,000 (eventually)

// Thread 2 - Withdrawal
int balance = getBalance(7774); // also reads 10,000 (T1 hasn't set yet!)
balance = balance - 5000; // computes 5,000
// ← CONTEXT SWITCH HAPPENS HERE
setBalance(7774, balance); // sets 5,000 - overwrites T1's result!

// Final balance: ₹5,000 instead of correct ₹10,000
// ₹5,000 is lost - data is inconsistent
```

Manual Flag-Based Locking (Naive Approach)

- **Idea:** use a boolean flag to prevent the second thread from entering the critical section

```
boolean flag = false; // shared flag

// Thread 1 - Deposit
while (flag) { Thread.sleep(20); } // wait if locked
flag = true; // lock
int balance = getBalance(7774);
balance = balance + 5000;
setBalance(7774, balance);
flag = false; // unlock

// Thread 2 - Withdrawal
while (flag) { Thread.sleep(20); } // wait if locked
flag = true;
int balance = getBalance(7774);
balance = balance - 5000;
setBalance(7774, balance);
flag = false;
```

- **Problem:** if the programmer forgets `flag = false`, Thread 2 waits forever → **starvation** or **deadlock**
- Java does not trust programmers to manage locking/unlocking manually → provides the `synchronized` keyword

The `synchronized` Keyword

- Java's built-in locking mechanism; automatically acquires and releases the lock
- Two forms:

Synchronized Block (partial critical section)

```
public void display() {  
    // statements 1 and 2 – not critical, all threads can execute simultaneously  
  
    synchronized (sharedObject) {  
        // statements 3, 4, 5 – critical section  
        // only ONE thread can be inside this block at a time  
        // T1 enters → T2 and T3 wait outside  
        // T1 exits → T2 enters → T3 waits, and so on  
    }  
  
    // statements 6-10 – not critical, all threads can execute simultaneously  
}
```

Synchronized Method (entire method is critical)

```
public synchronized void display() {  
    // ALL statements are critical  
    // No two threads can call this method on the same object simultaneously  
}
```

- **Vector vs. ArrayList**: **Vector** has all methods marked **synchronized**; **ArrayList** does not
 - Use **Vector** in multi-threaded environments; **ArrayList** in single-threaded for better performance

Shared Data Synchronization Example

```
class Data {
    int num = 0;
}

class IncrementThread extends Thread {
    Data objData;

    IncrementThread(Data d) {
        this.objData = d;
    }

    @Override
    public void run() {
        while (true) {
            synchronized (objData) {
                objData.num++;
                System.out.println("Increment: " + objData.num);
            }
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {}
        }
    }
}

class DecrementThread implements Runnable {
    Data objData;

    DecrementThread(Data d) {
        this.objData = d;
    }

    @Override
    public void run() {
        while (true) {
            synchronized (objData) {
                objData.num--;
                System.out.println("Decrement: " + objData.num);
            }
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {}
        }
    }
}

// In main:
Data objData = new Data();
IncrementThread inc = new IncrementThread(objData);
Thread dec = new Thread(new DecrementThread(objData));
inc.start();
dec.start();
```

```
// Without synchronized: num jumps to -1, -2, -1, -1 (inconsistent)
// With synchronized:   num alternates cleanly: 0→1→0→-1→0→1→0 (consistent)
```

`wait()` / `notify()` / `notifyAll()` — Inter-thread Communication

- Used when one thread needs to pause and wait for a signal from another thread
- Must **always** be called inside a `synchronized` block (otherwise throws `IllegalMonitorStateException`)

```
Object dummy = new Object(); // shared communicator object

// In ShowCircle thread's run():
synchronized (dummy) {
    dummy.wait(5000);
    // Pauses this thread for up to 5 seconds
    // If notified before 5 seconds, resumes immediately
    // If not notified within 5 seconds, resumes automatically
}

// In ShowRectangle thread's run():
if (yPos ≥ 550) {
    synchronized (dummy) {
        dummy.notify();
        // Wakes up the first thread waiting on dummy
        // dummy.notifyAll() would wake ALL waiting threads
    }
}
```

- `notify()` : wakes the **first** thread that called `wait()` on this object (FIFO order)
- `notifyAll()` : broadcasts to **all** waiting threads; whichever gains the lock first proceeds
- **Real-world analogy**: instructor tells students "wait, I'll be back in 5 minutes"; if instructor returns early, students resume; if instructor doesn't return in 5 minutes, students carry on anyway

`join()` — Waiting for a Thread to Die

- Makes the current thread pause until the target thread terminates

```
Thread rectangleThread = new Thread(showRectangle);
rectangleThread.start();

// In circle thread's run():
rectangleThread.join();
// Circle thread waits here until rectangleThread terminates
// Only then does circle thread begin executing

// With timeout:
rectangleThread.join(3000);
// Wait at most 3 seconds; if rectangleThread hasn't finished, proceed anyway
```

- Chaining multiple joins:

```
thread1.join(3000);    // wait up to 3s for thread1
thread2.join(2000);    // then wait up to 2s for thread2
// After both timeouts (or terminations), continue
```

- **Practical use:** rectangle thread breaks out of its loop when `yPos ≥ 300`, `run()` returns, thread dies; circle thread's `join()` unblocks and circle starts moving

GUI Threading Demonstration Summary

- **Single-threaded GUI:** clicking Start enters an infinite loop; clicking Close does nothing (main thread is blocked)
- **Multi-threaded GUI:** rectangle runs in its own daemon thread; closing the window terminates the worker thread (main window), JVM shuts down, daemon thread is killed
- **Correct approach:** use `runningStatus` flag so the rectangle thread exits cleanly when the window closes

Java Date API and Closing Guidance

Legacy `Date` Class

```
import java.util.Date;

Date currentDate = new Date();
System.out.println(currentDate);
// Output: Wed Sep 23 19:40:14 IST 2026
```

- Provides current system date and time
- Basic methods: `after()`, `before()`, `compareTo()`
- Largely superseded by the modern `LocalDateTime` API

`System.currentTimeMillis()`

```
long millis = System.currentTimeMillis();
System.out.println(millis);
// Output: number of milliseconds elapsed since 1st January 1970 (Unix epoch)
// e.g., 1758000000000
```

- Returns a `long` value representing elapsed milliseconds from the Unix epoch (1st Jan 1970)
- Useful for measuring elapsed time or generating timestamps

Java 8 `LocalDateTime` (JODA-inspired API)

- Introduced with Java 8; supports **localization** (date/time representation varies by region)
 - India: `DD/MM/YYYY`
 - USA: `MM/DD/YYYY`
- `LocalDateTime.now()` with a `ZoneId` for timezone-aware date and time:

```
import java.time.LocalDateTime;
import java.time.ZoneId;

// Current date and time in India (IST)
LocalDateTime indiaTime = LocalDateTime.now(ZoneId.of("Asia/Kolkata"));
System.out.println(indiaTime);
// Output: 2026-09-23T19:46:51.123

// Current date and time in Chicago (CST)
LocalDateTime chicagoTime = LocalDateTime.now(ZoneId.of("America/Chicago"));
System.out.println(chicagoTime);
// Output: 2026-09-23T09:17:xx.xxx

// Current date and time in New York (EST)
LocalDateTime newYorkTime = LocalDateTime.now(ZoneId.of("America/New_York"));
System.out.println(newYorkTime);
// Output: 2026-09-23T10:18:xx.xxx
```

- Creating a specific date and time manually:

```
LocalDateTime specific = LocalDateTime.of(2026, 9, 23, 19, 46);
// Year, Month, Day, Hour, Minute
```

- Internally, `LocalDateTime` fetches the current system time and applies the GMT offset for the given `ZoneId` to compute the local time for that region

JDBC Deferral

- Today is the **last Java session** for now
- JDBC (Java Database Connectivity) requires knowledge of databases first
- JDBC session will be scheduled approximately **15 days from now**, after the database module is complete
- All source code for every project covered in this module has been shared as a zip file (previously only screenshots were shared)

Prerequisites Before Web Java Begins

The following topics must be self-studied before the Web Java module starts:

- HTML** — mandatory; must be fully comfortable with it (available on W3Schools)
- Basic JavaScript** — fundamentals required

3. **XML** (Extensible Markup Language) — simple concept; can be finished in 30–45 minutes on W3Schools
4. **DTD** (Document Type Definition) — defines the structure/rules of an XML document
5. **XML Schema (XSD)** — a more powerful alternative to DTD for validating XML
6. **JSON** (JavaScript Object Notation) — json.org covers the entire spec in one page
 - Defines how to serialize a Java object into text for transmission over the internet
 - Used universally: Node.js, AJAX, React, REST APIs — everything uses JSON
 - Only 4–5 rules define the entire format

Web Java Roadmap (in order)

Servlets → JSP (JavaServer Pages) — covered in ~1 day → Hibernate (ORM) — initially confusing, becomes intuitive quickly → Spring Framework — 3–4 days → Advanced Spring → Spring Boot → REST Services / REST APIs → Microservices

- Final projects in Java will be built on: **Spring Boot + REST APIs + Microservices**
- Everything before Spring Boot is foundational groundwork ("kindergarten for Java")
- Web Java is significantly more intensive than core Java: concepts arrive rapidly, one after another
- The **last 3 days** of Web Java contain the most critical content; projects are built on those concepts

Java Core Concepts to Revise Before Web Java

The following must be crystal clear before starting Web Java (they are used extensively):

- Exception handling
- Upcasting and downcasting
- Lambda expressions
- Stream API (`filter` , `map` , `collect` , `groupingBy` , `partitioningBy`)
- Generics and wildcards
- Collections framework
- Multi-threading and synchronization

Final Advice

- Practice all assignments and exercises given during this module
- DSA and Database modules are coming next — use that time to revise Java thoroughly
- No need to re-learn from scratch; focused revision is sufficient
- Web Java will be fast-paced with heavy concept density; a weak Java foundation will cause serious problems

"Whatever you learn up to the last 3 days of Web Java is just the base — your projects are built on those last 3 days"

Chat with meeting transcript: <https://notes.granola.ai/t/5daa9c1f-1345-4ab2-81c4-a3d3f3e80058-00demib2>